UNIVERSITÉ DE LORRAINE | IUT Saint-Dié-des-Vosges

RÉSEAU COCAGNE

PROPOSITION TECHNIQUE

Septembre 2026

Équipe : Wendy Mechisso | Eunice Ligeiro

SOFTWARE ARCHITECTURE ÉVOLUTIONS

SOMMAIRE

01 - Vue d'ensemble de la solution

02 - Utilisateurs et fonctionnalités

03 - Architecture générale

04 - Architecture logicielle

05 - Stack technique

06 - Justification des choix techniques

07 - Organisation des données

08 - Fonctionnement de la solution

09 - Carte et itinéraires

10 - Calendrier des livraisons

11 - Architecture multi-sociétés

12 - Sécurité des données

13 - Hébergement et déploiement

14 - Robotique : retour autonome à la base

15 - Architecture matérielle envisagée

16 - Principe de navigation envisagé

17 - Interface PC et fonctionnement global

18 - Une solution modulaire et évolutive

VUE D'ENSEMBLE

RÉSEAU COCAGNE - NOUVEL OUTIL DE GESTION COMMERCIALE

CONSTAT

Outil actuel (Access) devenu limité face aux nouveaux besoins des Jardins.

SOLUTION PROPOSÉECURITÉ

Plateforme web mutualisée, paramétrable, Open Source et en mode SaaS.

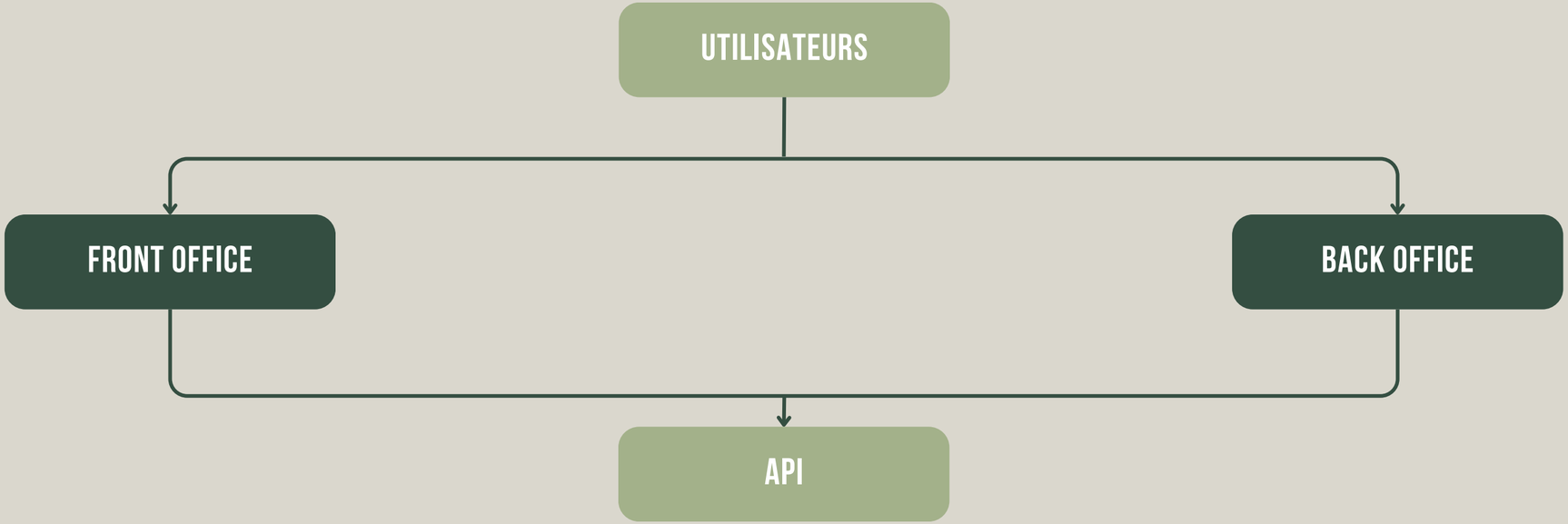
UTILISATEURS → APPLICATION WEB → API → BASE DE DONNÉES

OBJECTIFS

Centraliser | **Automatiser** | **Simplifier** | **Évoluer**

Objectifs techniques : simplicité d'usage, paramétrage fin, évolutivité de l'architecture

UNE SOLUTION ADAPTÉE AUX DIFFÉRENTS UTILISATEURS



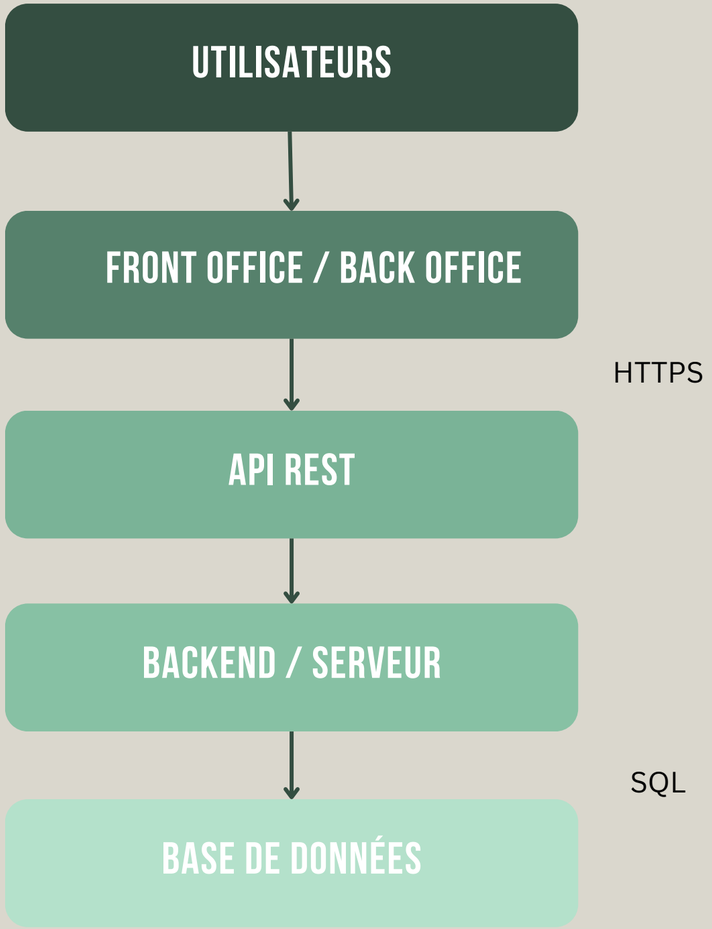
Front Office - clients / adhérents

- Consultation des produits et paniers
- Gestion de son abonnement
- Opérations liées aux achats

Back Office - équipes des Jardins

- Gestion des clients et abonnements
- Gestion des produits et livraisons
- Administration de la structure

ARCHITECTURE GÉNÉRALE DE LA SOLUTION



Back Office - équipes des Jardins

Chaque couche a un rôle précis, indépendant des autres.

Évolutivité

Les composants peuvent évoluer sans impacter l'ensemble.

Maintenance facilitée

Des développeurs des Jardins pourront reprendre le code.

ARCHITECTURE LOGICIELLE EN COUCHES

INTERFACE WEB	*Front Office / Back Office*
API REST	*Point d'entrée des échanges avec le client web*
LOGIQUE MÉTIER	*Règles de gestion : abonnements, livraisons, produits*
ACCÈS AUX DONNÉES	*Traduit la logique métier en requêtes SQL*
POSTGRESQL	*Stockage structuré et fiable des données*

TECHNOLOGIES PROPOSÉES

F

FRONTEND

REACT

Interface dynamique et composants réutilisables

B

BACKEND

PHP / SYMFONY

Architecture structurée, adaptée au développement PHP

A

API

REST / JSON

Communication standard entre les composants

C

CONTENEURISATION

DOCKER

Environnement reproductible, déploiement facilité

S

SERVEUR

LINUX

Fiabilité et coûts maîtrisés pour l'hébergement

F

VERSIONNEMENT

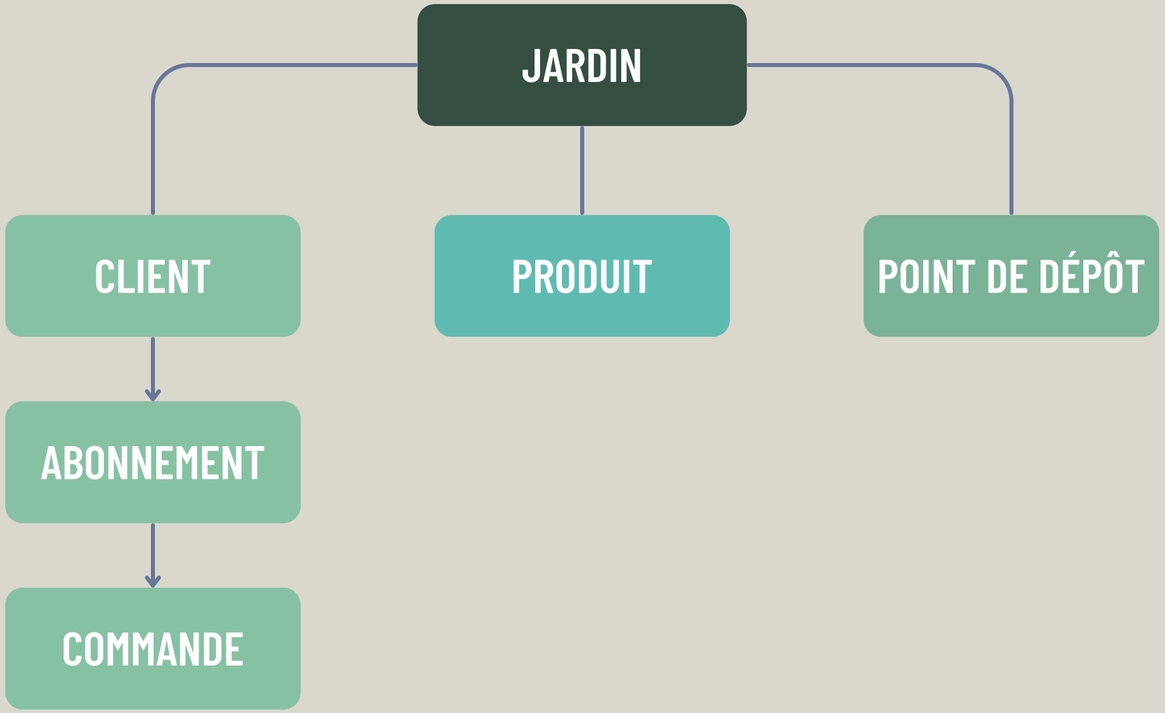
GIT

Travail collaboratif et suivi des évolutions

POURQUOI CES CHOIX ?

BESOIN DU CLIENT	RÉPONSE TECHNIQUE
SIMPLICITÉ	PHP / SYMFONY
ÉVOLUTIVITÉ	ARCHITECTURE MODULAIRE
ACCÈS UNIVERSEL	API REST
DONNÉES STRUCTURÉES	POSTGRESQL
DÉPLOIEMENT REPRODUCTIBLE	DOCKER
MAINTENANCE COLLABORATIVE	GIT

ORGANISATION DES DONNÉES



Principes

- Données structurées et normalisées
- Accès exclusivement via le backend
- Séparation logique des données par Jardin

Le modèle détaillé de la base de données fera l'objet du Rendu 6.

DU CLIC UTILISATEUR À LA BASE DE DONNÉES

1. *Utilisateur*
2. *Interface Web*
3. *Requête HTTPS*
4. *API REST*
5. *Logique métier*
6. *PostgreSQL*
7. *Résultat renvoyé*

SÉCURITÉ

HTTPS

COMMUNICATION

REST / JSON

DONNÉES

SQL / PostgreSQL

« Une architecture séparée permet de faire évoluer indépendamment l'interface, les services et les données. »

CARTE ET ITINÉRAIRES

VISUALISER LES POINTS DE DÉPÔT ET FACILITER L'ORGANISATION DES TOURNÉES

POINTS DE DÉPÔT → API REST → OSRM → LEAFLET + OPENSTREETMAP

CARTE — LEAFLET + OPENSTREETMAP

- Carte interactive
- Localisation des points de dépôt
- Marqueurs par catégorie
- Visualisation des tournées

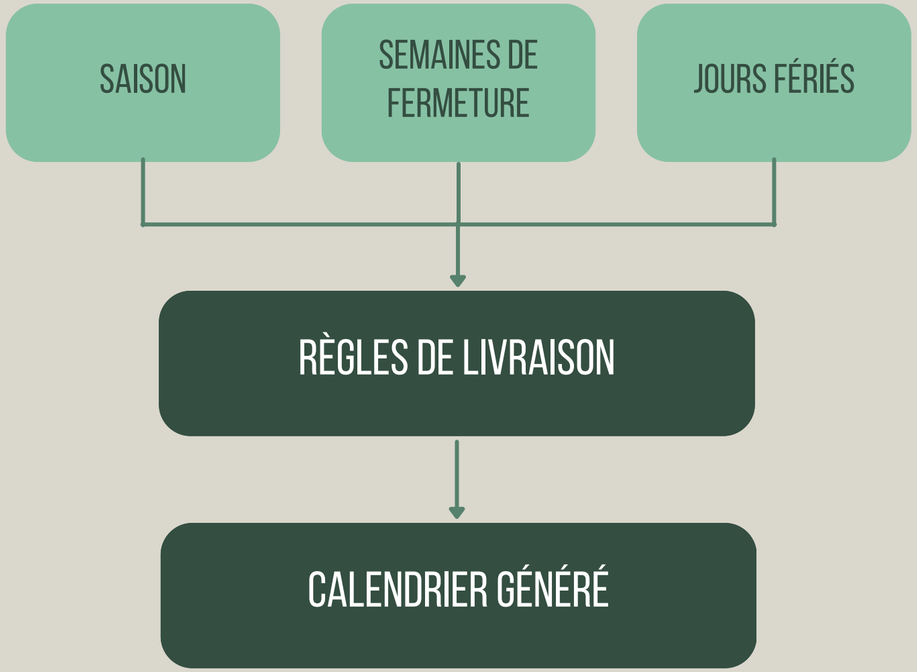
ITINÉRAIRES — OSRM

- Calcul des trajets
- Distances et durées
- Données OpenStreetMap
- Résultat affiché sur la carte

Une solution cartographique Open Source intégrée à l'application Web.

CALENDRIER DES LIVRAISONS

GÉNÉRER ET ADAPTER LES DATES DE LIVRAISON SELON LES RÈGLES DE CHAQUE JARDIN



LE SYSTÈME PREND EN COMPTE

- Saison / année
- Jour de livraison
- Tournée
- Fréquence de livraison
- Semaines de fermeture
- Jours fériés

SYMFONY

Application des règles métier

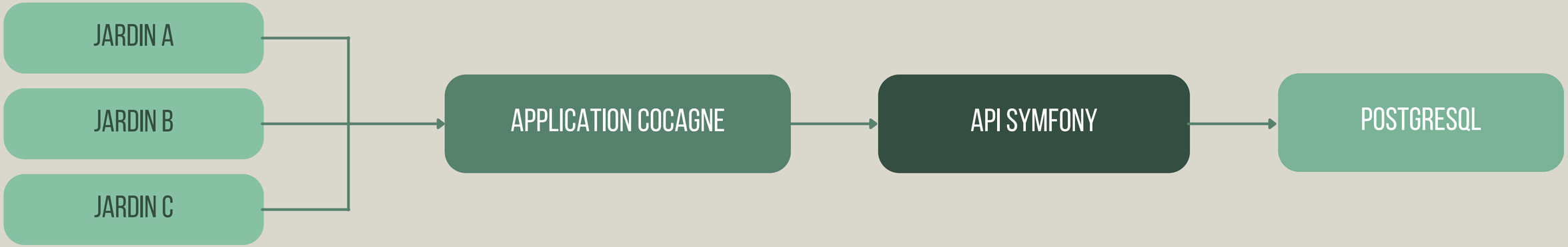
POSTGRESQL

Stockage des dates et calendriers

Un calendrier configurable selon le fonctionnement de chaque Jardin.

ARCHITECTURE MULTI-SOCIÉTÉS

UNE PLATEFORME COMMUNE, DES DONNÉES PROPRES À CHAQUE JARDIN



PLATEFORME MUTUALISÉE

Une seule application pour l'ensemble du Réseau.

ISOLATION DES DONNÉES

Chaque Jardin accède uniquement à ses propres données.

CONFIGURATION INDÉPENDANTE

Produits, dépôts, tournées et paramètres propres à chaque structure.

Une architecture SaaS centralisée tout en préservant l'indépendance de chaque Jardin.

SÉCURITÉ DES DONNÉES

PROTÉGER LES UTILISATEURS, LES ÉCHANGES ET LES DONNÉES

AUTHENTIFICATION

Accès à l'application par compte utilisateur.

RÔLES & AUTORISATIONS

Droits adaptés au profil et aux responsabilités.

MOTS DE PASSE

Stockage sécurisé sous forme de hash.

HTTPS

Communications chiffrées entre navigateur et serveur.

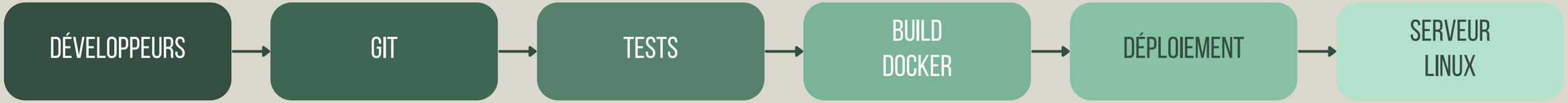
SAUVEGARDES

Sauvegardes régulières de la base de données.

Chaque utilisateur accède uniquement aux fonctionnalités et aux données qui lui sont autorisées.

HÉBERGEMENT ET DÉPLOIEMENT

UNE INFRASTRUCTURE CENTRALISÉE ET REPRODUCTIBLE



LINUX
Serveur centralisé

DOCKER
Environnement reproductible

GIT
Versionnement du code

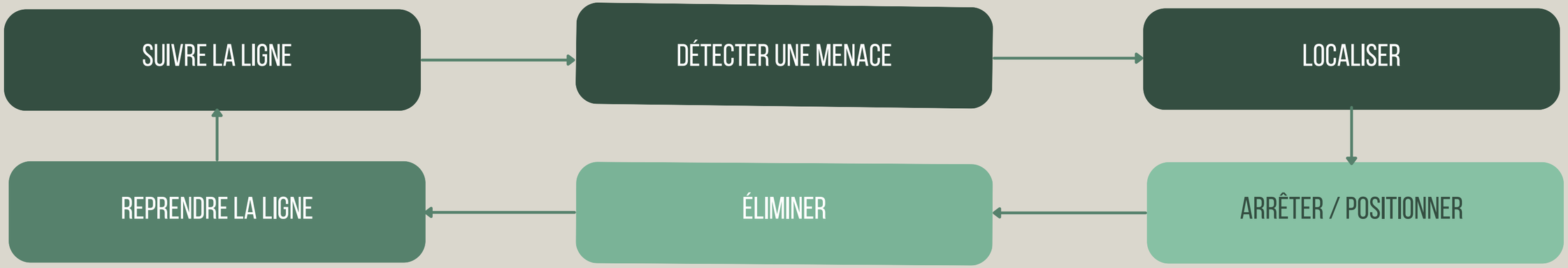
CI/CD
Tests et déploiement automatisés

MAINTENANCE
Mises à jour centralisées

Une infrastructure facilitant le déploiement, les mises à jour et la maintenance.

ROBOTIQUE 1 - OBJECTIF ET SCÉNARIO

SUIVRE UNE LIGNE DE CULTURE, IDENTIFIER LES MENACES, LES LOCALISER ET LES ÉLIMINER



NAVIGATION

Suivre automatiquement la ligne de culture.

MENACE

Détecter et déterminer sa position.

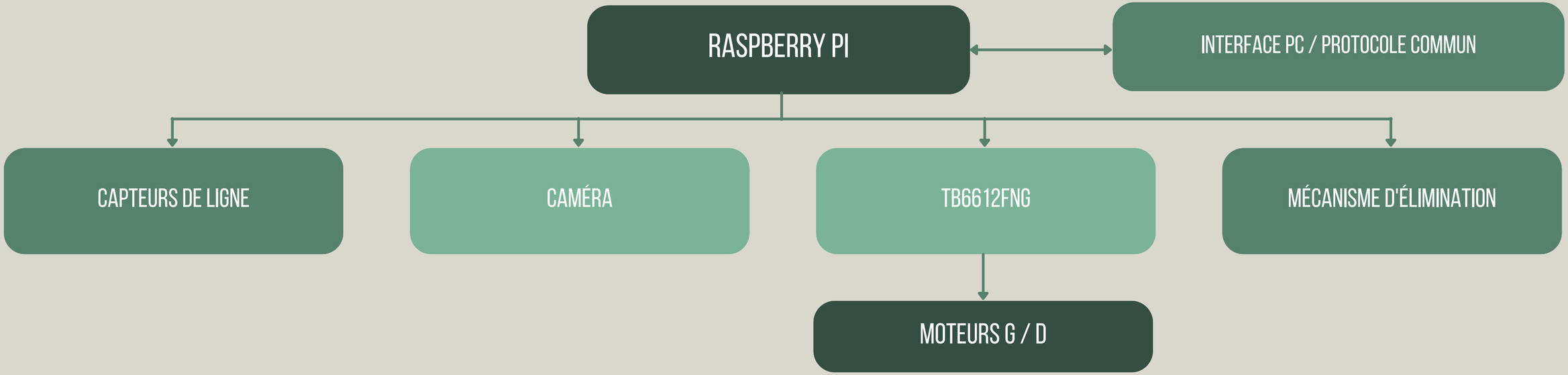
ACTION

Éliminer la menace puis poursuivre le parcours.

Scénario : suivre la ligne → détecter → localiser → arrêter/positionner → éliminer → reprendre la ligne.

ARCHITECTURE TECHNIQUE DU ROBOT

NAVIGATION, DÉTECTION DES MENACES, ÉLIMINATION ET SUPERVISION PC



NAVIGATION

Capteurs de ligne + moteurs

VISION

Caméra + détection / localisation

ACTION

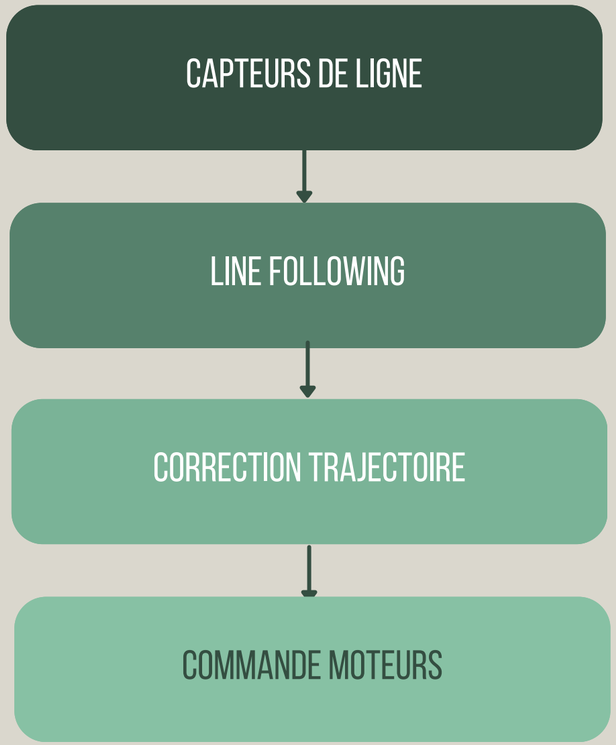
Mécanisme d'élimination

NAVIGATION & MONITORING

SUIVI DE LIGNE, MOTEURS ET CONTRÔLE DU DÉPLACEMENT

RÉALISATION ROBOTIQUE

- Contrôler les moteurs via TB6612FNG
- Commandes : avancer, reculer, gauche, droite, arrêt
- Lire les capteurs de suivi de ligne
- Développer le line following
- Corriger la trajectoire
- Gérer les états de navigation



INTERFACE PC

- START / STOP
- Mode automatique / manuel
- État du robot
- Valeurs des capteurs
- Vitesse

TESTS

Capteurs simulés, moteurs simulés et faux états avant l'intégration.

Navigation autonome et panneau de monitoring.

DÉTECTION & GESTION DES MENACES

CAMÉRA, TRAITEMENT D'IMAGE, LOCALISATION ET MÉCANISME D'ÉLIMINATION

CAMÉRA → CAPTURE IMAGE → DÉTECTION → LOCALISATION → ÉLIMINATION

TRAITEMENT D'IMAGE

- Capturer et traiter les images
- Détecter la culture et/ou les menaces
- Exemple : mauvaises herbes
- Déterminer la position dans l'image

ÉLIMINATION

- Commander le mécanisme d'élimination
- Vérifier le résultat si possible
- Garder la détection indépendante de la navigation

TESTS INDÉPENDANTS

- Images / vidéos enregistrées
- Faux état du robot
- Tester detection.py sans attendre le line following

Caméra + détection + localisation + actionneur + panneau de détection PC.

INTERFACE PC & COMMUNICATION

DEUX PANNEAUX COMPLÉMENTAIRES RELIÉS PAR UN PROTOCOLE COMMUN

PANNEAU NAVIGATION

- START / STOP
- Mode automatique / manuel
- État du robot
- Capteurs gauche / centre / droite
- Vitesse
- Commandes au robot

PROTOCOLE COMMUN

- START / STOP
- GET_STATUS
- ROBOT_STATUS
- THREAT_DETECTED
- ELIMINATE

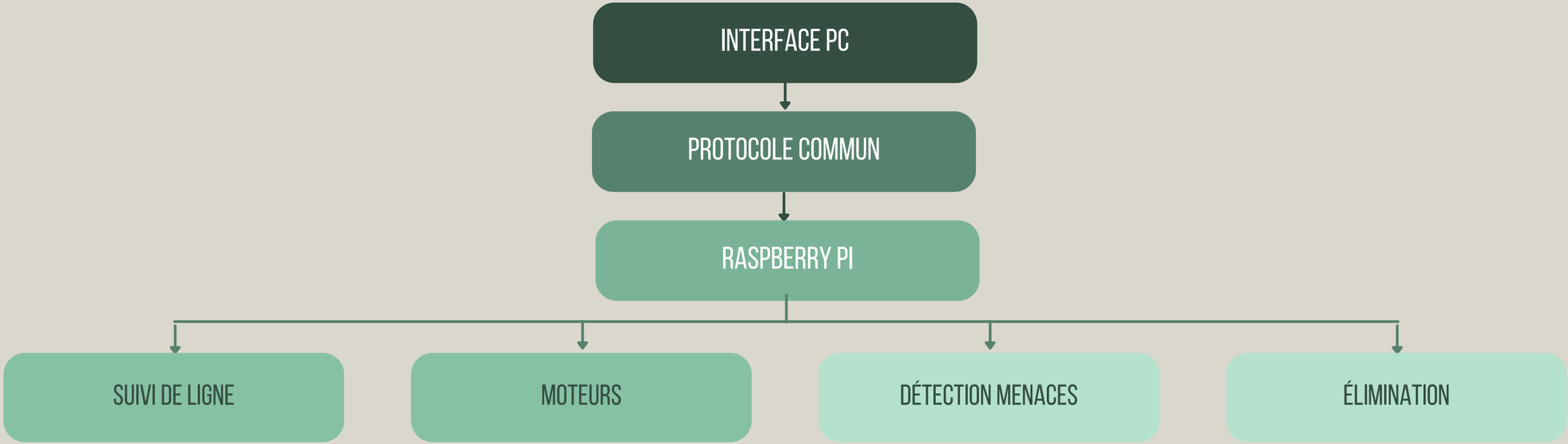
PANNEAU DÉTECTION

- Flux / image caméra
- Menace détectée et son type
- Position
- Nombre de menaces détectées
- État du mécanisme
- START / STOP DETECTION
- ELIMINATE si prévu

Le protocole commun permet de développer et tester navigation et détection indépendamment.

ARCHITECTURE GLOBALE — ROBOTIQUE 1

INTÉGRATION DE LA NAVIGATION, DE LA DÉTECTION DES MENACES ET DE L'ÉLIMINATION



SUIVRE → DÉTECTER → LOCALISER → ARRÊTER / POSITIONNER → ÉLIMINER → REPRENDRE

UNE SOLUTION MODULAIRE ET ÉVOLUTIVE

SYNTHÈSE ET JUSTIFICATION DES CHOIX TECHNIQUES

FRONT-END

React

Interface Web moderne

BACK-END

PHP / Symfony

Logique métier et API

DONNÉES

PostgreSQL

Stockage structuré

CARTOGRAPHIE

Leaflet + OpenStreetMap

Carte interactive

ITINÉRAIRES

OSRM

Calcul des trajets

INFRASTRUCTURE

Docker + Linux

Déploiement reproductible

ROBOTIQUE

Raspberry Pi 4 + Python / OpenCV

Suivi de ligne, détection et élimination des menaces

OPEN SOURCE

MODULAIRE

CENTRALISÉE

Une architecture adaptée aux besoins du Réseau Cocagne, avec une intégration robotique réalisée progressivement et validée par des tests

FIN

MERCI DE VOTRE ATTENTION